

★ Your membership will expire on April 14, 2026 [Reactivate membership](#)

Building Your Own Q&A Agent: A Simplified Guide



Haoyuan Zhang

Follow

8 min read · Mar 25, 2025



6



Steps to build and deploy your own Q&A Agent that can answer questions about your own dataset



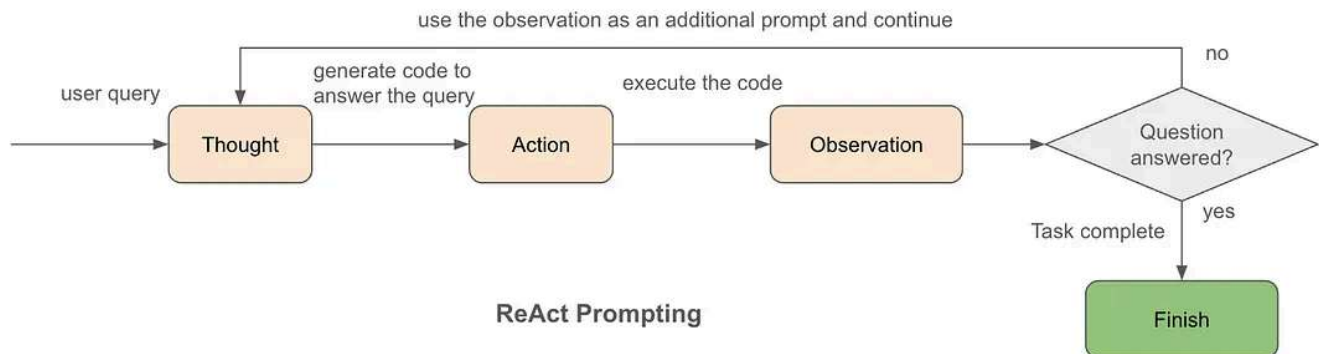
The full codes are in my github [here](#).

Background and Motivation

Part of data scientists' work often involves answering ad-hoc questions from stakeholders. These questions can range from simple data lookups to complex queries requiring aggregation and filtering.

This blog introduces a way to address this challenge: a ReAct-based Q&A LLM agent specifically designed to answer questions about your dataset. **ReAct** is a prompting technique that enables language models to document their reasoning process when answering questions. This is achieved by

generating a series of Thought, Action, and Observation steps, which improves transparency and makes the model's responses easier to understand and trust. By leveraging the power of Large Language Models (LLMs) and the ReAct prompting technique, this agent can automate the process of understanding and responding to ad-hoc data inquiries.



Motivation: The primary motivation behind this tool is to empower DS and stakeholders by streamlining the process of accessing and understanding data insights to reduce ad-hoc request burden and empower stakeholders with self-service analytics

Here are the key steps:

1. **Set up:** Set up your environment and load your dataset
2. **ReAct prompt:** Define a ReAct prompt containing model instructions, table schema, and few-shot examples to guide the model's reasoning.
3. **ReAct agent:** Create a ReAct agent using the ReAct class, which encapsulates the interaction with the Gemini model.
4. **Ask questions:** Interact with the agent by asking questions about your dataset. The agent will use its tools (search, execute, finish) to find answers.

5. Deployment

Setup

1. **Connecting to a runtime:** Connect to your runtime
2. **Import libraries**
3. **Setting up your API key:** You need to obtain an API key from the Google AI and configure the `google.generativeai` library with it. This allows you to access the Gemini model. If you don't have one yet, go to <https://ai.google.dev/gemini-api/docs/api-key> to create your API key.
4. **Testing the model:** A brief test with the `gemini-2.0-flash` model is included to ensure it's functioning correctly.
5. **Loading data:** Then loads a dataset called `developer_df` using an `f1` as our test example.

```
import pandas as pd
import numpy as np
import random
import string
import re
import google.generativeai as genai

YOUR_API_KEY = "" # Replace with your actual API key
genai.configure(api_key=YOUR_API_KEY)

model = genai.GenerativeModel('models/gemini-2.0-flash')
response = model.generate_content("Who is Google")
print(response.text[0:100])

del model
```

Load data

```

## Replace the following synthetic data with your own dataset.

random.seed(42) # Set the random seed for reproducibility.

def generate_random_string(length=10):
    """Generates a random string of specified length."""
    letters = string.ascii_letters
    return ''.join(random.choice(letters) for _ in range(length))

def generate_random_id(length=8):
    """Generates a random integer ID of specified length."""
    return random.randint(10**(length-1), (10**length)-1)

num_rows = 1000 # Number of rows to generate

data = {
    'store_id': [generate_random_id() for _ in range(num_rows)],
    'store_name': [generate_random_string() for _ in range(num_rows)],
    'region_code': [random.choice(["US", "CA", "UK", "DE", "FR", "JP", "AU"]) for _ in range(num_rows)],
    'store_type': [random.choice(['Supermarket', 'Convenience Store']) for _ in range(num_rows)],
    'num_products': [random.randint(1, 50) for _ in range(num_rows)],
    'num_customers_last_28d': [random.randint(10, 10000) for _ in range(num_rows)],
    'num_customers_last_180d': [random.randint(100, 100000) for _ in range(num_rows)],
    'num_customers_last_365d': [random.randint(1000, 1000000) for _ in range(num_rows)],
    'revenues_last28d': [random.randint(100, 1000000) for _ in range(num_rows)],
    'revenues_last180d': [random.randint(1000, 10000000) for _ in range(num_rows)],
    'revenues_last365d': [random.randint(10000, 100000000) for _ in range(num_rows)]
}

store_df = pd.DataFrame(data)
store_df.head(5)

```

The ReAct Prompt

```

# Define model instructions for ReAct prompting
# The model instruction was borrowed from the ReAct paper with a few minor adjustments

model_instructions = """
Solve a question answering task with interleaving Thought, Action, Observation steps.
Only use the results from the table provided.

```

Thought can reason about the current situation,

Observation is understanding relevant information from an Action's output and

Action can be of three types:

- (1) entity, which searches the exact entity on table scheme from table `store\_df` and returns the column or columns of interested. We already have a dataframe ca. If you cannot find it, you will return some similar columns to search the information.
- (2) code, which execute the python code without printing function, assign the final result to a variable.
- (3) answer, which returns the answer from the execution step and finishes the task.

```
"""
```

```
# Define table schema for the developer dataframe
```

```
table_schema = """
```

```
Here is the table schema for table `store_df`, these description which can help you understand the data.
The schema description is:
```

Column Name	Description
`store_id`	The unique identifier of the store.
`store_name`	The name of the store.
`region_code`	The region code where the store is located.
`store_type`	The type of store, such as retail, wholesale, etc.
`num_products`	The total number of products in the store.
`num_customers_last_28d`	The number of customers in the last 28 days.
`num_customers_last_180d`	The number of customers in the last 180 days.
`num_customers_last_365d`	The number of customers in the last 365 days.
`revenues_last28d`	The total revenue generated in the last 28 days.
`revenues_last180d`	The total revenue generated in the last 180 days.
`revenues_last365d`	The total revenue generated in the last 365 days.

```
"""
```

```
# Define few-shot examples for in-context learning
```

```
examples = """
```

```
Here are an example.
```

```
Question 1
```

```
How much United State stores made in the last 28d?
```

```
Thought 1
```

```
I need to find store that are in US and their corresponding revenue value in the last 28 days.
```

```
## Action 1:
```

```
import pandas as pd
```

```
__result__ = store_df[store_df['region_code'] == 'United State']['revenues_last28d'].sum()
```

```

## Thought 2:
The code cannot find United State. I will try to find a store in US instead

## Action 2:

import pandas as pd
__result__ = store_df[store_df['region_code'] == 'US']['revenues_last28d'].sum

## Thought 3:
The code successfully retrieved the 28d revenue of store in the US.

## Action 4:
Store in the US made $78,808,584 last 28 days.
"""

# Combine instructions, schema, and examples into the final ReAct prompt
ReAct_prompt = model_instructions + table_schema+ examples

```

The ReAct Agent Pipeline

Here we build a pipeline to Agent with the ReAct-prompted Gemini model.

In this code, the ReAct pipeline is implemented using the ReAct class. It defines the tools (search, execute, finish) and manages the interaction with the Gemini model. The `__call__` method handles the multi-turn conversation and function calling. This allows users to ask questions about the provided `developer_df` dataset and receive answers along with the reasoning process.

```

# Define the ReAct class for interacting with the Gemini model

class ReAct:
    def __init__(self, model: str, ReAct_prompt: str):
        """

```

Initializes the ReAct agent, enabling the Gemini model to understand and respond to a 'Few-shot ReAct prompt'. This is achieved by mimicking the 'function calling' technique, which allows the model to generate both reasoning steps and specific actions in an interleaved fashion.

Args:

model: name to the model.
ReAct_prompt: ReAct prompt.

"""

```
self.model = genai.GenerativeModel(model)
self.chat = self.model.start_chat(history=[])
self.should_continue_prompting = True
self._search_history: list[str] = []
self._search_urls: list[str] = []
self._prompt = ReAct_prompt
```

@property

```
def prompt(self):
    return self._prompt
```

@classmethod

```
def add_method(cls, func):
    setattr(cls, func.__name__, func)
```

@staticmethod

```
def clean(text: str):
    """Helper function for responses."""
    text = text.replace("\n", " ")
    return text
```

@ReAct.add_method

```
def search(self, query: str):
```

"""

Performs search on `query` via a given dataframe.

Args:

query: Search parameter to query the dataframe.

Returns:

observation: Summary of the search finding for `query` if found.

"""

```
query = query.strip()
```

```
try:
```

```
    ## instruct the model to generate python code based on the query
```

```
    observation = self.model.generate_content("""
```

```
        Question: write a python code without any explanation on question: {}.
```

```
        Please do not name the final output.
```

```
        Only return the value of the output without print function.
```

```

    Answer:
    """.format(query))

    observation = observation.text
    result = eval(observation.replace('``python', '').replace('```', ''))

    ## keep search history
    self._search_history.append(query)
    self._search_results.append(result)
except:
    observation = f'Could not find [{query}]'

    return observation

@ReAct.add_method
def execute(self, code_phrase: str):
    """
    Execute `code_phrase` from search and return the result.

    Args:
        phrase: The code snippet to look up the values of intested.

    Returns:
        code_result: Result after executing the `code_phrase` .
    """

    code_result = {}
    try:
        exec(code_phrase.replace('``python', '').replace('```', ''), globals(), c
    except:
        code_result = f'Could not execute code[{code_phrase}]'
    return code_result

@ReAct.add_method
def finish(self, _):
    """
    Stops the question-answering process when the model generates a ``
    token. This is achieved by setting the `self.should_continue_prompting` flag
    to `False`, which signals to the agent that the final answer has been reached.
    """

    self.should_continue_prompting = False

@ReAct.add_method
def __call__(self, user_question, max_calls: int=10, **generation_kwargs):
    """
    Starts multi-turn conversation with the LLM models, using function calling
    to interact with external tools.

    Args:
        user_question: The initial question from the user.

```



```

max_calls: The maximum number of calls to the model before ending the
            conversation.
generation_kwargs: Additional keyword arguments for text generation,
                    such as temperature and max_output_tokens. See
                    `genai.GenerativeModel.GenerationConfig` for details.

Raises:
    AssertionError: if max_calls is not between 1 and 10
    """

# set a higher max_calls for more complex task.
assert 0 < max_calls <= 10, "max_calls must be between 1 and 10"

if len(self.chat.history) == 0:
    model_prompt = 'Based on the dataset from store_df, ' + self.prompt + user_q
else:
    model_prompt = 'Based on the dataset from store_df, ' + user_question

# stop_sequences for the model to imitate function calling
callable_entities = ['', '', '']
generation_kwargs.update({'stop_sequences': callable_entities})

self.should_continue_prompting = True
for idx in range(max_calls):

    self.response = self.chat.send_message(
        content=[model_prompt],
        generation_config=generation_kwargs,
        stream=False)

    for chunk in self.response:
        print(chunk.text.replace("tool_code", '').replace("`", ''), end='\n')

    response_cmd = self.chat.history[-1].parts[-1].text

    try:
        cmd = re.findall(r'<(.*?)>', response_cmd)[-1]
        query = response_cmd.split(f'<{cmd}>')[-1].strip()

        # call to appropriate function
        observation = self.__getattr__(cmd)(query)

        if not self.should_continue_prompting:
            break

        stream_message = f"\nObservation {idx + 1}\n{observation}"

        # send function's output as user's response to continue the conversation
        model_prompt = f"<{cmd}>{query}{cmd}>'s Output: {stream_message}"
    except (IndexError, AttributeError) as e:

```

```
model_prompt = "Please try to generate as instructed by the prompt."  
  
return
```

Ask Questions

```
gemini_ReAct_chat = ReAct('models/gemini-2.0-flash', ReAct_prompt=ReAct_prompt)  
  
#@title Q&A { vertical-output: false, form-width: "70%" }  
#@markdown Run this to query the Q&A Agent:  
#@markdown Examples:  
#@markdown 1) How many stores in the US?  
#@markdown 2) Which store have the most products?  
#@markdown 3) The top 3 stores with the most customers last year?  
  
query = "How many stores in the US?" #@param {type:"string"}  
temperature = 0.0 # @param { type: "slider", min: 0, max: 1, step: 0.1}  
gemini_ReAct_chat(query, temperature=temperature)
```

Summary

- By grounding LLM with provided dataset via ReAct prompting, we minimize hallucinations.
- The model's Thought-Action-Observation traces provide a transparent, step-by-step view of its reasoning, boosting user trust.
- This agent is easily adaptable to your own datasets by modifying the schema and examples. Detailed schemas and examples significantly improve query accuracy.

Next Step — Deployment

Make your LLM Agent accessible to stakeholders by deploying it. This allows them to directly leverage its capabilities. For rapid development of internal AI applications, consider using Mesop (see [go/mesop](https://go.mesop)), an open-source Python UI framework. I've deployed my agent at [here](#). You can also checkout my code in Github [here](#).

Agents

Llm Applications

Data Science

Artificial Intelligence

React

**Written by Haoyuan Zhang**

2 followers · 1 following

Follow

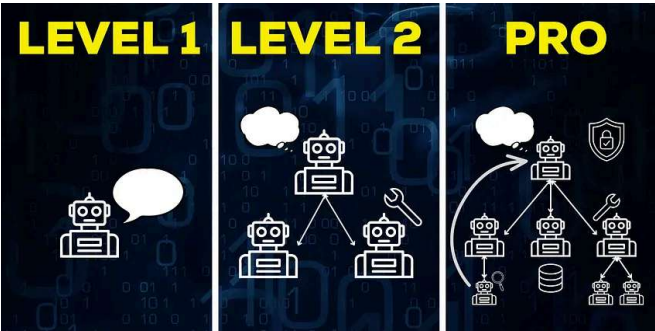
No responses yet



Jingdan Chen him/he

What are your thoughts?

Recommended from Medium



 In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

★ Dec 6, 2025  5K  187  ...

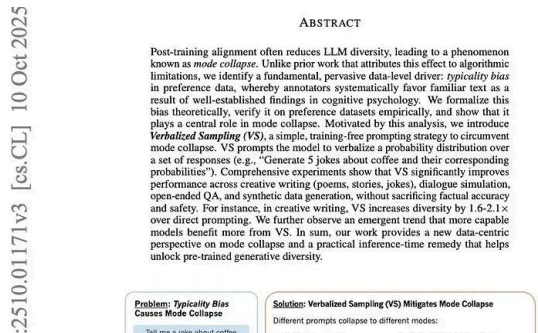


 In Artificial Corner by The PyCoach

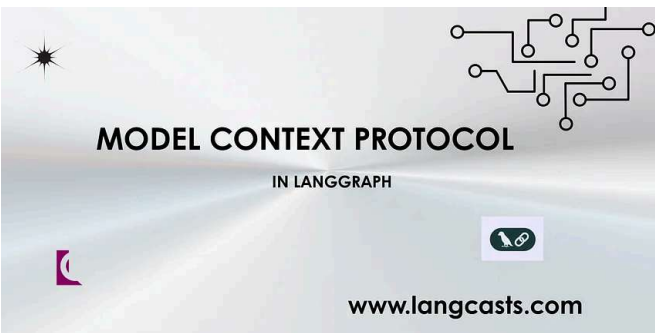
The Best AI Tools for 2026

If you're going to learn a new AI tool, make sure it's one of these

★ Dec 1, 2025  5.4K  318  ...



 In Generative AI by Adham Khaled



 In AI Engineering BootCa... by Damilola Oyedunm...

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2x...

A Beginner's Guide to Using MCP with LangGraph

If you've been following along with the LangGraph series, you already know how...

Open in app ↗

≡

Medium

🔍 Search

✍️ Write

🔔

J

★





In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

★ Jan 29

👏 6.9K

💬 66

💬 ...



 unicodeveloper

10 Must-Have Skills for Claude (and Any Coding Agent) in 2026

The definitive guide to agent skills that change how Claude Code, Cursor, Gemini CL...

Mar 9

👏 933

💬 14

💬 ...

See more recommendations

https://medium.com/@howardhyzhang/building-your-own-q-a-agent-a-simplified-guide-82475bec8b5d

13/13